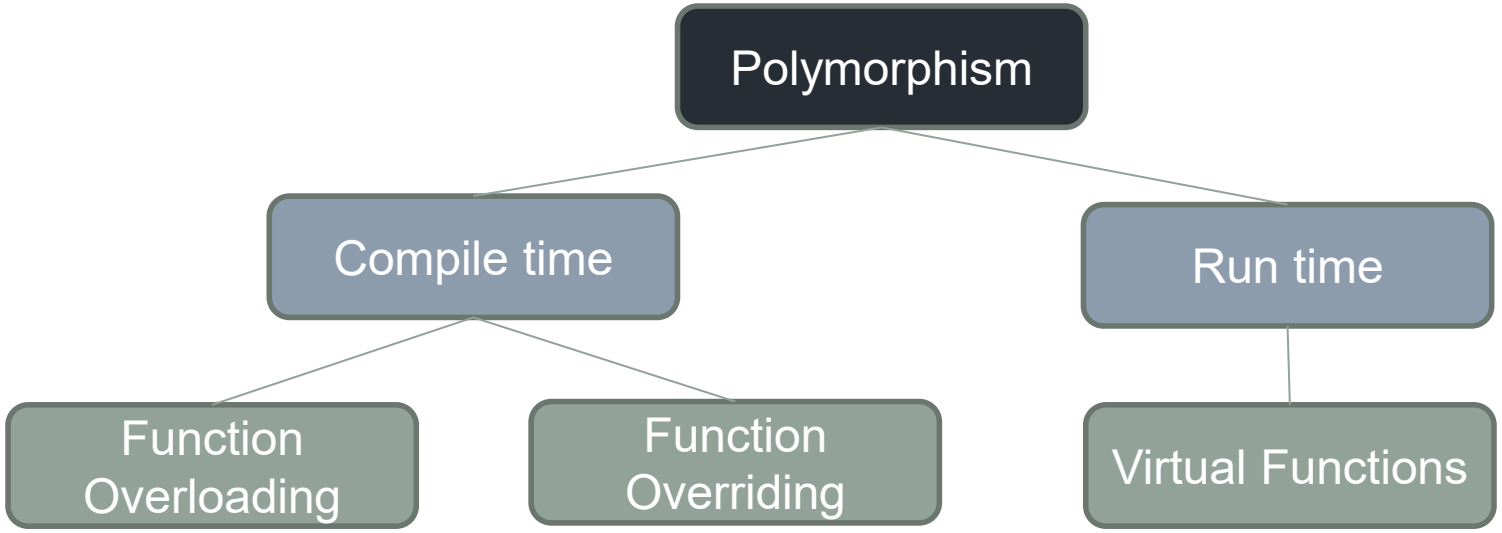
POLYMORPHISM, VIRTUAL FUNCTIONS & ABSTRACT CLASSES

Polymorphism

- The **word polymorphism** means having multiple forms.
- The **term Polymorphism** gets derived from the **Greek word** poly + morphos, where **poly** means many and **morphos** means forms.
- Polymorphism in programming means: a function same as another in **name** but with different behavior.



Polymorphism

```
DemoClass d;

d.add(3, 7);          // int, int
d.add(1, 9, 8);       // int, int, int
d.add(3.14, 5);        //double, int
```

```
class DemoClass
{
    public:
    int add(int a, int b)
    {
        return a + b;
    }

    int add(int x, int y, int z)
    {
        return x + y + z;
    }

    int add(double pi, int x)
    {
        return (int)pi + x;
    }
};
```

Compile time Polymorphism

Polymorphism

- You can pass an object of a derived class to a formal parameter of the base class type

```
class baseClass
{
public:
    void print();
    baseClass(int u = 0);

private:
    int x;
};

class derivedClass: public baseClass
{
public:
    void print();
    derivedClass(int u = 0, int v = 0);

private:
    int a;
};
```

```
void baseClass::print()
{
    cout << "In baseClass x = " << x << endl;
}

baseClass::baseClass(int u)
{
    x = u;
}

void derivedClass::print()
{
    cout << "In derivedClass ***: ";
    baseClass::print();
    cout << "In derivedClass a = " << a << endl;
}

derivedClass::derivedClass(int u, int v)
                : baseClass(u)
{
    a = v;
}
```

Polymorphism Cont.…

```
void callPrint(baseClass& p)
{
    p.print();
}
```

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In baseClass x = 3
```

```
int main()
{
    baseClass one(5);                          //Line 1
    derivedClass two(3, 15);                   //Line 2

    one.print();                               //Line 3
    two.print();                               //Line 4

    cout << "*** Calling the function "
         << "callPrint  ***" << endl;          //Line 5

    callPrint(one);                            //Line 6
    callPrint(two);                            //Line 7

    return 0;
}
```

Polymorphism (cont..)

- For both statements (Lines 6 and 7), member function `print` of `baseClass` was executed
  - Because the binding of `print`, in the body of `callPrint`, occurred at compile time
- <u>Compile-time binding</u>: the necessary code to call a specific function is generated by the compiler
  - Also known as **static binding or early binding**

Virtual Functions

- How can we avoid this problem?
  - Virtual functions (reserved word `virtual`)
- Virtual function: binding occurs at program execution time, not at compile time
  - This kind of binding is called run-time binding
- Run-time binding: The compiler does not generate code to call a specific function; it generates information to enable the run-time system to generate specific code for the function call
  - Also known as **dynamic binding**

Virtual Functions (contd…)

```
class baseClass
{
public:
    virtual void print();        //virtual function
    baseClass(int u = 0);

private:
    int x;
};

class derivedClass: public baseClass
{
public:
    void print();
    derivedClass(int u = 0, int v = 0);

private:
    int a;
};
```

Sample Run

```
In base class  x = 5
In derived class ***; In base class  x = 3
In derived class a = 15
*** Calling the function callPrint() ***
In base class  x = 5
In derived class ***; In base class  x = 3
In derived class a = 15
```

```
void callPrint(baseClass& p)
{
    p.print();
}
```

```
int main()
{
    baseClass one(5);                          //Line 1
    derivedClass two(3, 15);                   //Line 2

    one.print();                               //Line 3
    two.print();                               //Line 4

    cout << "*** Calling the function "
         << "callPrint  ***" << endl;          //Line 5

    callPrint(one);                            //Line 6
    callPrint(two);                            //Line 7

    return 0;
}
```

Type compatibility, the simplest case

Each new class constitutes a new type of data. Each object constructed based on such a class is like a value of the new type.

This means that any two objects may (or may not) be compatible in the sense of their types.

Look at the very simple example on the right

```
#include <iostream>
using namespace std;
class Cat {
public:
    void MakeSound(void) { cout << "Meow! Meow!" << endl; }
};
class Dog {
public:
    void MakeSound(void) { cout << "Woof! Woof!" << endl; }
};
int main(void) {
    Cat *a_cat = new Cat();
    Dog *a_dog = new Dog();
    a_cat -> MakeSound();
    a_dog -> MakeSound();
    return 0;
}
```

Type compatibility, the simplest case

- The **objects** of the **Cat class** are not compatible with the **objects** of the **Dog class**, although the **structure** of both classes is **identical**.
- Neither of the following assignments is valid, and both will cause a compiler error:

```
a_dog = a_cat;
a_cat = a_dog;
```

- The **Dog** and **Cat** classes have nothing in common in terms of inheritance.
- They are both completely independent.
- So, we can say that **objects derived from classes that lie in different branches of the inheritance tree are always incompatible.**

Type compatibility, more complex case

We have rebuilt our example. We still have the two previous classes, Dog and Cat, but we have put them into the same inheritance tree.

```
#include <iostream>
#include <string>
using namespace std;
class Pet {
protected:
    string Name;
public:
    Pet(string n) { Name = n; }
    void Run(void) { cout << Name << ": I'm running" << endl; }
};
class Dog : public Pet {
public:
    Dog(string n) : Pet(n) {};
    void MakeSound(void) { cout << Name << ": Woof! Woof!" << endl; }
};
class Cat : public Pet {
public:
    Cat(string n) : Pet(n) {};
    void MakeSound(void) { cout << Name << ": Meow! Meow!" << endl; }
};
int main(void) {
    Pet a_pet("pet");
    Cat a_cat("Tom");
    Dog a_dog("Spike");
    a_pet.Run();
    a_dog.Run(); a_dog.MakeSound();
    a_cat.Run(); a_cat.MakeSound();
    return 0;
}
```

Type compatibility, more complex

- **Objects** of a **subclass** are compatible with a **superclass reference** (this is called **upcasting** and is allowed **implicitly**).

- **Objects** of a **superclass** are not directly compatible with a **subclass reference** (this is called **downcasting** and requires an **explicit cast** and is only safe if the object is of the subclass type).

Type compatibility, more complex

- This means that: You **can** do the following:

```
a_pet = new Dog("Huckleberry");
a_pet -> Run();
```

- You are **not allowed** to do the following:

~~a_dog = new Pet("Strange pet")~~;

Classes and Virtual Destructors

- Classes with pointer member variables should have the destructor
- A destructor can be designed to deallocate storage for dynamic objects
- If a derived class object is passed to a formal parameter of the base class type, the destructor of the base class executes
- Regardless of whether the object is passed by reference or by value

- **Solution:** use a virtual destructor (base class)

```
class A
{
    public:
        void fun1(){ cout<<"Fun1 of Class A"<<endl; }
        virtual void fun2(){ cout<<"Fun2 of Class A"<<endl; }
        ~A(){ cout<<"Destructor of A"<<endl; }
};

class B: public A
{
    private:
        int *arr;
    public:
        B(int sz)
        {
            arr = new int[sz];
        }
        void fun1(){ cout<<"Fun1 of Class B"<<endl; }
        void fun2(){ cout<<"Fun2 of Class B"<<endl; }
        ~B()
        {
            cout<<"Destructor of B"<<endl;
            delete arr;
        }
};
```

Output

```
Fun1 of Class A
Fun2 of Class B
Destructor of A
```

```
int main()
{
    A *p;
    p = new B(5);

    p->fun1();
    p->fun2();

    delete p;

    return 0;
}
```

```
class A
{
    public:
        void fun1(){ cout<<"Fun1 of Class A"<<endl; }
        virtual void fun2(){ cout<<"Fun2 of Class A"<<endl; }
        virtual ~A(){ cout<<"Destructor of A"<<endl; }
};

class B: public A
{
    private:
        int *arr;
    public:
        B(int sz)
        {
            arr = new int[sz];
        }
        void fun1(){ cout<<"Fun1 of Class B"<<endl; }
        void fun2(){ cout<<"Fun2 of Class B"<<endl; }
        ~B()
        {
            cout<<"Destructor of B"<<endl;
            delete arr;
        }
};
```

```
int main()
{
    A *p;
    p = new B(5);

    p->fun1();
    p->fun2();

    delete p;

    return 0;
}
```

Output

```
Fun1 of Class A
Fun2 of Class B
Destructor of B
Destructor of A
```

```
class A
{
public:
    void fun1(){cout<<"Fun1 of class A\n";}
    virtual void fun2(){cout<<"Fun2 of class A\n";}
    void fun3(){cout<<"Fun3 of class A\n";}
    ~A(){cout<<"Destructor of A\n";}
};
class B:public A
{
public:
    virtual void fun1(){cout<<"Fun1 of class B\n";}
    void fun2(){cout<<"Fun2 of class B\n";}
    void fun3(){cout<<"Fun3 of class B\n";}
    ~B(){cout<<"Destructor of B\n";}
};
class C:public B
{
public:
    void fun1(){cout<<"Fun1 of class C\n";}
    void fun2(){cout<<"Fun2 of class C\n";}
    virtual void fun3(){cout<<"Fun3 of class C\n";}
    ~C(){cout<<"Destructor of C\n";}
};
void OutFun(B & objParameter)
{
    objParameter.fun1();
    objParameter.fun2();
    objParameter.fun3();
}
void main()
{
    C objC;
    OutFun(objC);
    B *p=new C;
    delete p;
}
```

Virtual functions for class A:

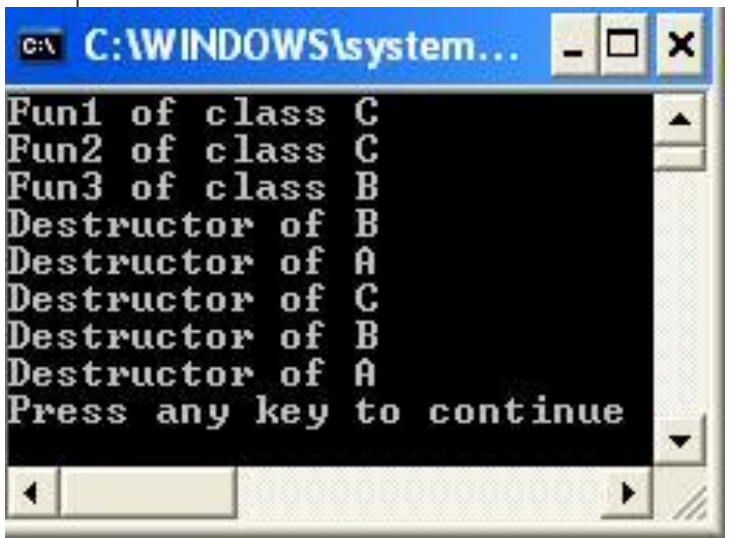
fun2

Virtual functions for class B:

fun1 and fun2

Virtual functions for class C:

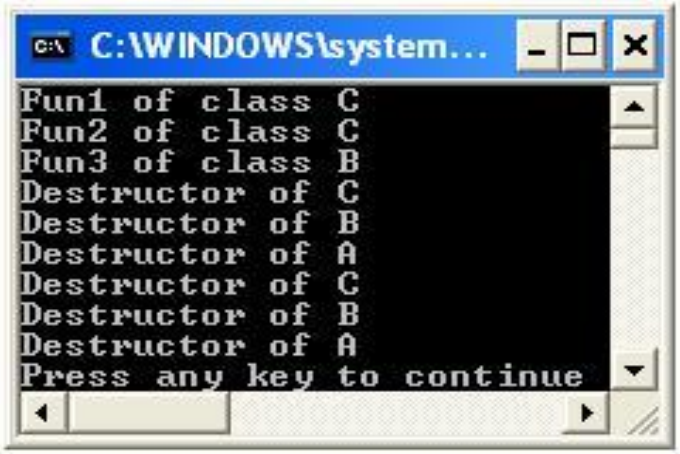
fun1 and fun2 and fun3



Classes and Virtual Destructors

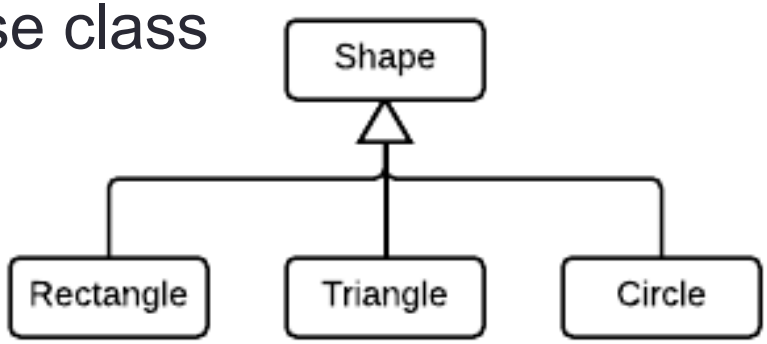
- The **virtual destructor** of a base class automatically makes the destructor of a derived class virtual
- After executing the destructor of the derived class, the destructor of the base class executes
- If a base class contains virtual functions, make the destructor of the base class virtual

```
class A
{
public:
    void fun1(){cout<<"Fun1 of class A\n";}
    virtual void fun2(){cout<<"Fun2 of class A\n";}
    void fun3(){cout<<"Fun3 of class A\n";}
    virtual~A(){cout<<"Destructor of A\n";}
};
class B:public A
{
public:
    virtual void fun1(){cout<<"Fun1 of class B\n";}
    void fun2(){cout<<"Fun2 of class B\n";}
    void fun3(){cout<<"Fun3 of class B\n";}
    ~B(){cout<<"Destructor of B\n";}
};
class C:public B
{
public:
    void fun1(){cout<<"Fun1 of class C\n";}
    void fun2(){cout<<"Fun2 of class C\n";}
    virtual void fun3(){cout<<"Fun3 of class C\n";}
    ~C(){cout<<"Destructor of C\n";}
};
void OutFun(B & objParameter)
{
    objParameter.fun1();
    objParameter.fun2();
    objParameter.fun3();
}
void main()
{
    C objC;
    OutFun(objC);
    B *p=new C;
    delete p;
}
```



Abstract Classes and Pure Virtual Functions

- Through inheritance, we can derive new classes without designing them from scratch
- Derived classes inherit existing members of the base class, can add their own members, and redefine or override public and protected member functions
- Base class can contain functions that you would want each derived class to implement
- Base class may contain functions that may not have meaningful definitions in the base class



Abstract Classes and Pure Virtual Functions (continued)

```
class shape
{
public:
    virtual void draw();
      //Function to draw the shape.

    virtual void move(double x, double y);
      //Function to move the shape at the position
      //(x, y).

    .
    .
    .
};
```

```
virtual void draw() = 0;
virtual void move(double x, double y) = 0;
```

- To make them **pure virtual functions**:

Abstract Classes – Shapes

```
class Shape
{
    public:
        virtual void draw() = 0;
        virtual double area() = 0;
};

class Rectangle: public Shape
{
    private:
        double length, width;
    public:
        Rectangle(double l, double w):length(l), width(w){}

        void draw()
        {
            cout<<"Drawing a rectangle"<<endl;
        }

        double area()
        {
            return length*width;
        }
};
```

Output

```
Drawing a rectangle
Area: 25.37
Drawing a Circle
Area: 32.1536
```

```
class Circle: public Shape
{
    private:
        double radius;
    public:
        Circle(double r): radius(r) {}

        void draw()
        {
            cout<<"Drawing a Circle"<<endl;
        }

        double area()
        {
            return 3.14*radius*radius; //pi*r*r
        }
};


int main()
{
    Rectangle *r = new Rectangle(4.3, 5.9);
    Circle *c = new Circle(3.2);
    r->draw();
    cout<<"Area: "<<r->area()<<endl;
    c->draw();
    cout<<"Area: "<<c->area()<<endl;

    return 0;
}
```

Abstract Classes – Shapes

```
class Shape
{
    public:
        virtual void draw() = 0;
        virtual double area() = 0;
};

class Rectangle: public Shape
{
    private:
        double length, width;
    public:
        Rectangle(double l, double w):length(l), width(w){}

        void draw()
        {
            cout<<"Drawing a rectangle"<<endl;
        }

        double area()
        {
            return length*width;
        }
};
```

Output

```
Drawing a rectangle
Area: 25.37
Drawing a Circle
Area: 32.1536
```

```
class Circle: public Shape
{
    private:
        double radius;
    public:
        Circle(double r): radius(r) {}

        void draw()
        {
            cout<<"Drawing a Circle"<<endl;
        }

        double area()
        {
            return 3.14*radius*radius; //pi*r*r
        }
};


int main()
{
    Rectangle *r = new Rectangle(4.3, 5.9);
    Circle *c = new Circle(3.2);

    Shape *s[] = {r, c};
    for (int i = 0; i < 2; i++)
    {
        s[i]->draw();
        cout<<"Area: "<<s[i]->area()<<endl;
    }
    return 0;
}
```

Abstract Classes and Pure Virtual Functions (continued)

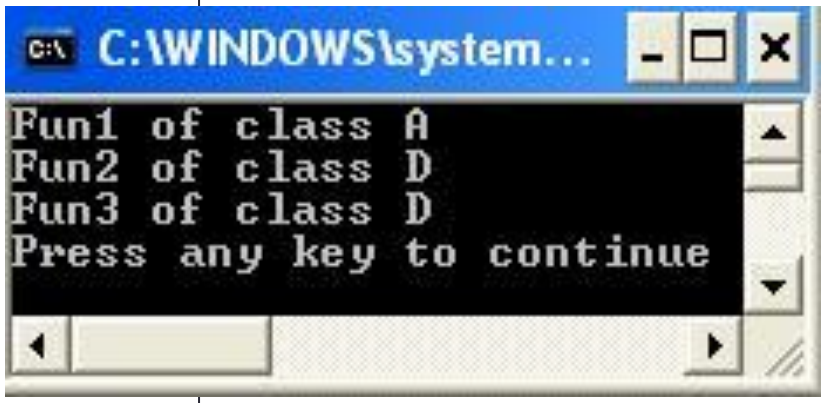
- Abstract class: contains one or more pure virtual functions

```
class shape
{
public:
    virtual void draw() = 0;
      //Function to draw the shape. Note that this is a
      //pure virtual function.
    virtual void move(double x, double y) = 0;
      //Function to move the shape at the position
      //(x, y). Note that this is a pure virtual
      //function.
    .
    .
    .

};
```

← You cannot create objects of an abstract class

Abstract Classes and Pure Virtual Functions (continued)

- If we derive a `rectangle` from a `shape`, and want to make it a non-abstract class, we must provide the definitions of the pure virtual functions of its base class
- Note that an abstract class can contain instance variables, constructors, and functions that are not pure virtual
- The class must provide the definitions of constructors/ functions that are not pure virtual

```
class A              //Abstarct
{
public:
    void fun1(){cout<<"Fun1 of class A\n";}
    virtual void fun2(){cout<<"Fun2 of class A\n";}
    virtual void fun3()=0;
};
class B:public A //Abstract...why????
{
public:
    void fun1(){cout<<"Fun1 of class B\n";}
    void fun2(){cout<<"Fun2 of class B\n";}
};
class D:public B //Non Abstract
{
public:
    void fun1(){cout<<"Fun1 of class D\n";}
    void fun2(){cout<<"Fun2 of class D\n";}
    void fun3(){cout<<"Fun3 of class D\n";}
};
//void OutFun(A obj)        illegal
// A OutFun ()              illegal
void OutFun(A & objParameter)
{
    objParameter.fun1();
    objParameter.fun2();
    objParameter.fun3();
}
void main()
{
    //A obj    B  obj     illegal
    D obj;
    OutFun(obj);
}
```